

Relational Diagram:-

step 1) Convert strong entities

- 1) User(user_id pk, last_name, first_name, email unique, phone_no unique, password, role, created_at)
- 2) Product(product_id pk, name, description, price, stock)
- 3) Seller(seller_id pk, name, email unique, phone_no unique)
- 4) Category(category_id pk, category_name)
- 5) Order(order_id pk, order_date, status, total_amount)
- 6) Payment(payment_id pk, payment_method, payment_status, payment_date)
- 7) Cart(cart_id pk, created_at)

Step 2) Convert weak entities:-

Rule : weak entity gets: Owners primary key as foreign key, its partial key, composite primary key.

- 1) Address(weak entity, address owned by users)

address(user_id fk, address_id, street, city, state, pincode,
primary key(user_id, address_id))

- 2) Review(user_id fk, product_id fk, rating, comment, review_date, primary
key(user_id, product_id))

Converting one to one relationships:-

user -- cart(1:1)

Cart(cart_id pk, created_at, user_id fk unique)

order -- payment(1:1)

Payment(payment_id pk, payment_method, payment_status, payment_date, order_id fk
unique)

Converting one to Many relationships(1:N):-

users -- orders(1:N)

Order(order_id pk, order_date, status, total_amount, user_id fk)

user -- address(1:N)

address(user_id fk, address_id, street, city, state, pincode,

primary key(user_id, address_id))

seller -- product(1:N)

Product(product_id pk, name, description, price, stock, seller_id fk)

category -- category(1:N)

Category(category_id pk, category_name, parent_category_id fk)

step 5) Convert many to many (M:N) relationships

rule: Create a junction table with PK of both entities and composite primary key.

1) Product -- category(M:N):-

product_category(product_id fk, category_id fk, primary key(product_id, category_id))

2) Order -- product(M:N):-

order_item(product_id fk, order_id fk, quantity, price_at_time, primary key(order_id, product_id))

3) product -- cart(M:N)

cart_item(product_id fk, cart_id fk, quantity, primary key(product_id, cart_id))

-- sql queries to build you ecommerce db

-- step 0 create database

create database if not exists Ecommerce_db;

use Ecommerce_db;

-- step 1 strong entities to table

create table Users(user_id int primary key auto_increment,
first_name varchar(225), last_name varchar(225),
email varchar(225) unique,
phone_no varchar(13) unique,
password varchar(225),
role enum("Customer", "Seller", "Admin"),
created_at timestamp default current_timestamp);

create table Seller(seller_id int primary key
, constraint FK_Seller_user foreign key(seller_id) references users(user_id)
on delete cascade on update cascade);

create table Category(category_id int primary key auto_increment,
category_name varchar(225), parent_category_id int,

```
constraint FK_Category_parent_cat foreign key(parent_category_id) references
category(category_id) on update cascade on delete set null);
```

```
create table Product(product_id int primary key auto_increment,
name varchar(225),
description text,
price decimal(9,2),
stock int,
seller_id int,
constraint FK_Product_seller foreign key(seller_id) references seller(seller_id)
on delete cascade on update cascade);
```

```
create table orders(order_id int primary key auto_increment,
order_date timestamp default current_timestamp,
status varchar(30),
total_amount decimal(9,2),
user_id int,
constraint FK_users_orders foreign key(user_id) references users(user_id)
on delete cascade on update cascade);
```

```
create table Cart(cart_id int primary key auto_increment,
created_at timestamp default current_timestamp,
user_id int unique,
constraint FK_User_cart foreign key(user_id) references users(user_id) on delete
cascade on update cascade);
```

```
create table Payment(payment_id int primary key auto_increment,
payment_method varchar(225),
payment_status varchar(225),
payment_date timestamp default current_timestamp,
order_id int unique,
constraint FK_User_payment foreign key(order_id) references orders(order_id)
on delete cascade on update cascade);
```

```
-- creating weak entities
create table address(user_id int, address_id int,
street varchar(225),
city varchar(225),
state varchar(225),
pincode varchar(6),
primary key(user_id, address_id),
constraint FK_Users_addres foreign key(user_id) references users(user_id) on delete
cascade on update cascade);
```

```
create table Review(user_id int, product_id int,
rating int check(rating >= 1 or rating <= 10),
comment text,
review_date timestamp default current_timestamp,
primary key(user_id, product_id),
```

```
constraint FK_User_review foreign key(user_id) references users(user_id) on delete
cascade on update cascade,
constraint FK_Product_review foreign key(product_id) references product(product_id)
on delete
cascade on update cascade);
```

```
-- many to many relationship junction tables
create table order_item(product_id int, order_id int,
    quantity int,
    price_at_time decimal(9,2),
    primary key(order_id, product_id),
    constraint FK_Order_product foreign key(product_id) references product(product_id)
on delete
cascade on update cascade,
constraint FK_order_item foreign key(order_id) references orders(order_id) on
delete
cascade on update cascade);
```

```
create table cart_item(product_id int, cart_id int,
    quantity int,
    primary key(product_id, cart_id),
    constraint FK_cart_item_product foreign key(product_id) references
product(product_id) on delete
cascade on update cascade,
constraint FK_cart_item_cart foreign key(cart_id) references cart(cart_id) on
delete
cascade on update cascade);
```

```
create table product_category(product_id int, category_id int,
    primary key(product_id, category_id),
    constraint FK_product_category_product foreign key(product_id) references
product(product_id) on delete
cascade on update cascade,
constraint FK_product_category_category foreign key(category_id) references
category(category_id) on delete
cascade on update cascade);
```